

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ PyCaret 소개

□ PyCaret을 활용한 분류

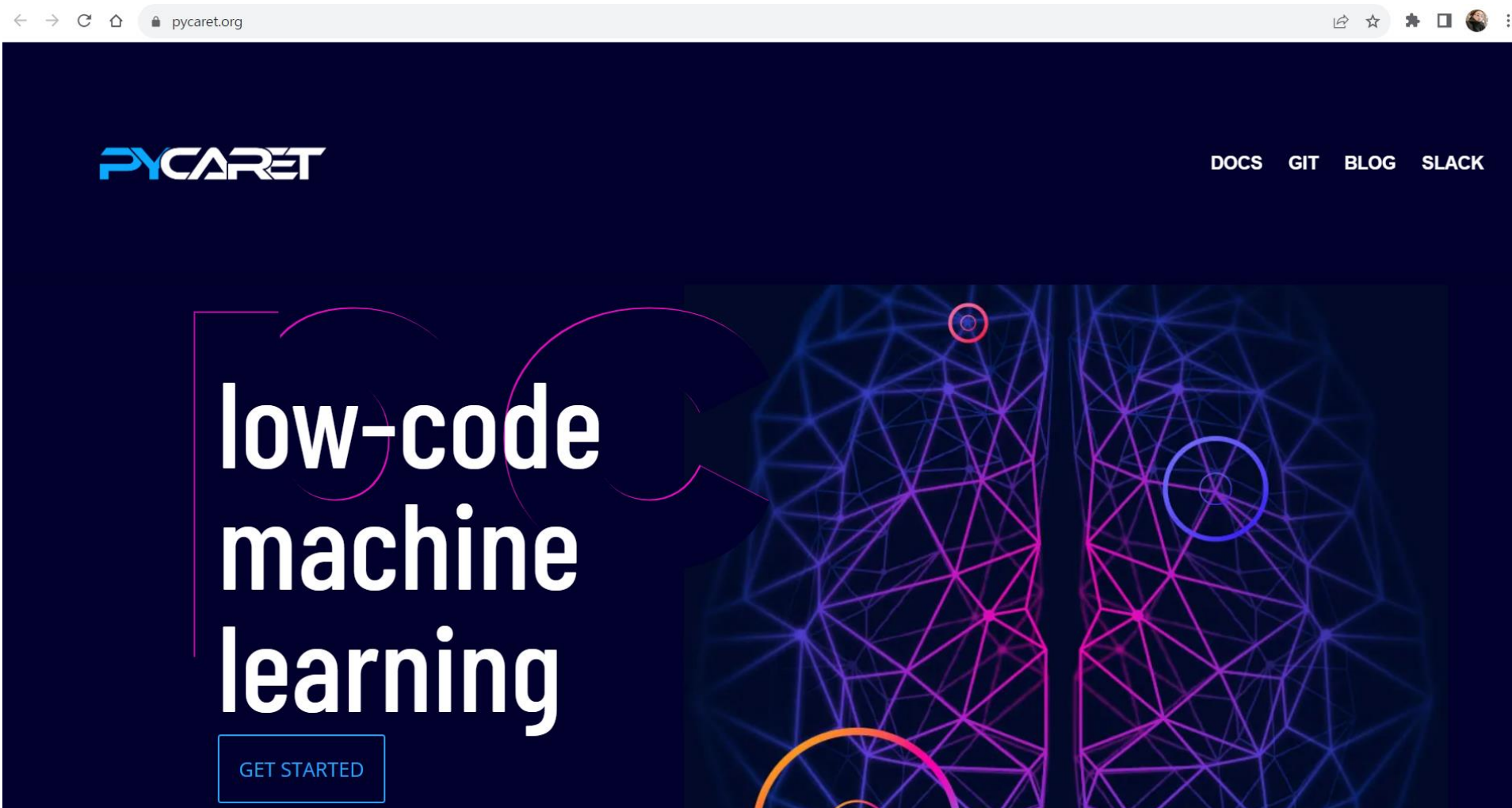
□ PyCaret을 활용한 회귀

AI/BigData 인텐시브 과정

Section 1. PyCaret

Section 1-1. PyCaret 설치

PyCaret 소개



PyCaret 소개

```
(base) C:\Users\Cheolwon>conda activate py3_11_9
```

```
(py3_11_9) C:\Users\Cheolwon> pip install pycaret
```

설치하는데 5분정도 걸립니다.

AI/BigData 인텐시브 과정

Section 1. PyCaret

Section 1-2. PyCaret을 활용한 분류

데이터 불러오기

```
In [1]: 1 import pandas as pd
        2 from pycaret.classification import *
```

```
In [2]: 1 df = pd.read_csv("../data/wine_data.csv")
```

```
In [3]: 1 df
```

```
Out[3]:
```

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

178 rows × 14 columns

setup

- 데이터 전처리
- session_id: 실험 ID
- 트레이닝, 테스트 데이터 분할

In [7]:

타겟 변수 설정 실험 ID

```
1 exp1 = setup(data=df, target='class', session_id=123)
```

	Description	Value
0	Session id	123
1	Target	class
2	Target type	Multiclass
3	Original data shape	(178, 14)
4	Transformed data shape	(178, 14)
5	Transformed train set shape	(124, 14)
6	Transformed test set shape	(54, 14)
7	Numeric features	13
8	Preprocess	True
9	Imputation type	simple
10	Numeric imputation	mean
11	Categorical imputation	mode
12	Fold Generator	StratifiedKFold
13	Fold Number	10
14	CPU Jobs	-1
15	Use GPU	False
16	Log Experiment	False
17	Experiment Name	clf-default-name
18	USI	b2e5

compare_models(): 모형 비교

```
In [8]: 1 best_model = compare_models()
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
et	Extra Trees Classifier	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0310
lda	Linear Discriminant Analysis	0.9917	1.0000	0.9917	0.9933	0.9917	0.9874	0.9883	0.0070
qda	Quadratic Discriminant Analysis	0.9846	1.0000	0.9846	0.9872	0.9843	0.9765	0.9781	0.0060
nb	Naive Bayes	0.9840	0.9990	0.9840	0.9873	0.9839	0.9758	0.9775	0.0050
rf	Random Forest Classifier	0.9840	0.9990	0.9840	0.9876	0.9841	0.9760	0.9777	0.0350
ridge	Ridge Classifier	0.9833	0.0000	0.9833	0.9868	0.9830	0.9745	0.9764	0.0060
lightgbm	Light Gradient Boosting Machine	0.9763	0.9981	0.9763	0.9812	0.9763	0.9642	0.9667	0.0530
lr	Logistic Regression	0.9429	0.9941	0.9429	0.9556	0.9422	0.9137	0.9205	0.4410
dt	Decision Tree Classifier	0.9282	0.9416	0.9282	0.9459	0.9259	0.8885	0.8986	0.0050
gbc	Gradient Boosting Classifier	0.9205	0.9731	0.9205	0.9367	0.9182	0.8768	0.8861	0.0380
ada	Ada Boost Classifier	0.9199	0.9834	0.9199	0.9387	0.9155	0.8752	0.8871	0.0160
knn	K Neighbors Classifier	0.6455	0.8498	0.6455	0.6275	0.6133	0.4531	0.4782	0.2030
svm	SVM - Linear Kernel	0.5872	0.0000	0.5872	0.4815	0.4949	0.3753	0.4639	0.0090
dummy	Dummy Classifier	0.4038	0.5000	0.4038	0.1633	0.2325	0.0000	0.0000	0.0050

tune_model: 하이퍼 파라미터 수정

In [9]: `1 tuned_model = tune_model(best_model)`

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
2	0.9231	0.9904	0.9231	0.9385	0.9231	0.8850	0.8929
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Mean	0.9923	0.9990	0.9923	0.9938	0.9923	0.9885	0.9893
Std	0.0231	0.0029	0.0231	0.0185	0.0231	0.0345	0.0321

Fitting 10 folds for each of 10 candidates, totalling 100 fits

Original model was better than the tuned model, hence it will be returned. NOTE: The display metrics are for the tuned model (not the original one).

모형 평가

In [10]:

1 evaluate_model(tuned_model)

Plot Type:	Pipeline Plot	Hyperparameters	AUC	Confusion Matrix	Threshold
	Precision Recall	Prediction Error	Class Report	Feature Selecti...	Learning Curve
	Manifold Learni...	Calibration Cur...	Validation Curve	Dimensions	Feature Importa...
	Feature Importa...	Decision Bounda...	Lift Chart	Gain Chart	Decision Tree
	KS Statistic Pl...				



예측

In [11]:

1 predict_model(tuned_model)

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Extra Trees Classifier	0.9815	1.0000	0.9815	0.9825	0.9815	0.9720	0.9725

Out[11]:

	Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
19	13.64	3.10	2.56	15.200000	116	2.70	3.03	0.17	1.66	5.10	0.96	3.36	845	0
28	13.87	1.90	2.80	19.400000	107	2.95	2.97	0.37	1.76	4.50	1.25	3.40	915	0
58	13.72	1.43	2.50	16.700001	108	3.40	3.67	0.19	2.04	6.80	0.89	2.87	1285	0
102	12.34	2.45	2.46	21.000000	98	2.56	2.11	0.34	1.31	2.80	0.80	3.38	438	1
48	14.10	2.02	2.40	18.799999	103	2.75	2.92	0.32	2.38	6.20	1.07	2.75	1060	0

최종 모형 저장

```
In [12]: 1 final_model = finalize_model(tuned_model)
```

```
In [13]: 1 save_model(final_model, 'wine_model')
```

Transformation Pipeline and Model Successfully Saved

```
Out[13]: (Pipeline(memory=Memory(location=None),
           steps=[('numerical_imputer',
                  TransformerWrapper(exclude=None,
                                     include=['Alcohol', 'Malic', 'Ash',
                                             'Alcalinity', 'Magesium',
                                             'Phenols', 'Flavanoids',
                                             'Nonflavanoids', 'Proanthocyanins',
                                             'Color', 'Hue', 'Dilution',
                                             'Proline'],
                                     transformer=SimpleImputer(add_indicator=False,
                                                                copy=True,
                                                                fill_value=None,
                                                                missing_values=nan,
                                                                strategy='m...
                  ExtraTreesClassifier(bootstrap=False, ccp_alpha=0.0,
                                       class_weight=None, criterion='gini',
                                       max_depth=None, max_features='sqrt',
                                       max_leaf_nodes=None, max_samples=None,
                                       min_impurity_decrease=0.0,
                                       min_samples_leaf=1, min_samples_split=2,
                                       min_weight_fraction_leaf=0.0,
                                       n_estimators=100, n_jobs=-1,
                                       oob_score=False, random_state=123,
                                       verbose=0, warm_start=False))),
           verbose=False),
          'wine_model.pkl')
```

저장된 모형 불러오기

```
In [14]: 1 loaded_model = load_model('wine_model')
```

Transformation Pipeline and Model Successfully Loaded

```
In [24]: 1 new_data = df.iloc[20:40,0:-1]
2 predictions = predict_model(loaded_model, data=new_data)
```

```
In [25]: 1 predictions
```

Out[25]:

Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	prediction_label	prediction_score
16.000000	126	3.00	3.17	0.24	2.10	5.65	1.09	3.71	780	0	1.0
18.600000	102	2.41	2.41	0.25	1.98	4.50	1.03	3.52	770	0	1.0
16.600000	101	2.61	2.88	0.27	1.69	3.80	1.11	4.00	1035	0	1.0
17.799999	95	2.48	2.37	0.26	1.46	3.93	1.09	3.63	1015	0	1.0
20.000000	96	2.53	2.61	0.28	1.66	3.52	1.12	3.82	845	0	1.0
25.000000	124	2.63	2.68	0.47	1.92	3.58	1.13	3.20	830	0	1.0

AI/BigData 인텐시브 과정

Section 1. PyCaret

Section 1-3. PyCaret을 활용한 회귀

데이터 불러오기

```
In [26]: 1 import pandas as pd
          2 from pycaret.regression import *
```

```
In [27]: 1 df = pd.read_csv("./data/boston_house_prices.csv")
```

```
In [28]: 1 df
```

Out[28]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

데이터 불러오기

```
In [30]: 1 exp_reg = setup(data=df, target='MEDV', session_id=123)
```

	Description	Value
0	Session id	123
1	Target	MEDV
2	Target type	Regression
3	Original data shape	(506, 14)
4	Transformed data shape	(506, 14)
5	Transformed train set shape	(354, 14)
6	Transformed test set shape	(152, 14)
7	Numeric features	13
8	Preprocess	True
9	Imputation type	simple
10	Numeric imputation	mean
11	Categorical imputation	mode
12	Fold Generator	KFold
13	Fold Number	10
14	CPU Jobs	-1
15	Use GPU	False
16	Log Experiment	False
17	Experiment Name	reg-default-name
18	USI	4203

compare_models(): 모형 비교

```
In [31]: 1 best_model = compare_models()
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
et	Extra Trees Regressor	2.0378	9.4958	2.9283	0.8884	0.1298	0.1000	0.0310
gbr	Gradient Boosting Regressor	2.1606	9.2764	2.9793	0.8833	0.1422	0.1113	0.0170
rf	Random Forest Regressor	2.2316	10.6310	3.1448	0.8673	0.1447	0.1129	0.0400
lightgbm	Light Gradient Boosting Machine	2.3305	11.5722	3.2969	0.8566	0.1482	0.1157	0.0430
ada	AdaBoost Regressor	2.8789	15.9344	3.8667	0.8055	0.1803	0.1517	0.0160
dt	Decision Tree Regressor	2.9307	19.6150	4.2538	0.7368	0.1909	0.1455	0.0080
ridge	Ridge Regression	3.2793	22.8402	4.6440	0.7300	0.2460	0.1664	0.0060
lr	Linear Regression	3.3006	22.7209	4.6402	0.7295	0.2526	0.1668	0.4250
lar	Least Angle Regression	3.3057	22.7377	4.6421	0.7293	0.2557	0.1672	0.0060
br	Bayesian Ridge	3.3158	23.2769	4.6862	0.7258	0.2453	0.1673	0.0070
lasso	Lasso Regression	3.6288	27.3444	5.0719	0.6780	0.2578	0.1743	0.2120
en	Elastic Net	3.6621	27.5698	5.1018	0.6742	0.2562	0.1743	0.0070
huber	Huber Regressor	4.0827	36.8851	5.9312	0.5365	0.2931	0.2080	0.0100
omp	Orthogonal Matching Pursuit	4.7560	46.4786	6.7186	0.4260	0.3682	0.2508	0.0070
knn	K Neighbors Regressor	4.8507	47.4264	6.7447	0.4226	0.2611	0.2269	0.0110
llar	Lasso Least Angle Regression	6.7170	86.4359	9.1940	-0.0478	0.3946	0.3689	0.0060
dummy	Dummy Regressor	6.7170	86.4359	9.1940	-0.0478	0.3946	0.3689	0.0070
par	Passive Aggressive Regressor	8.7856	130.0484	11.1739	-0.7478	0.4661	0.4557	0.0070

tune_model: 하이퍼 파라미터 수정

```
In [32]: 1 tuned_model = tune_model(best_model)
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	2.6595	13.4674	3.6698	0.8493	0.1394	0.0988
1	1.9068	6.7518	2.5984	0.8542	0.1191	0.1022
2	2.5426	9.5300	3.0871	0.9048	0.1732	0.1542
3	3.0106	23.9678	4.8957	0.7637	0.2017	0.1597
4	2.5438	10.9426	3.3080	0.9072	0.1524	0.1308
5	2.2067	9.1330	3.0221	0.8301	0.1552	0.1207
6	1.6454	4.2093	2.0517	0.9376	0.0933	0.0788
7	3.0894	34.0583	5.8359	0.6875	0.2059	0.1324
8	2.0147	6.7065	2.5897	0.8801	0.1588	0.1282
9	2.2593	10.9488	3.3089	0.8733	0.1439	0.0972
Mean	2.3879	12.9716	3.4367	0.8488	0.1543	0.1203
Std	0.4442	8.6909	1.0773	0.0705	0.0325	0.0246

Fitting 10 folds for each of 10 candidates, totalling 100 fits

Original model was better than the tuned model, hence it will be returned. NOTE: The display metrics are for the tuned model (not the original one).

모형 평가

```
In [33]: 1 evaluate_model(tuned_model)
          2
```

Plot Type:

Pipeline Plot

Hyperparameters

Residuals

Prediction Error

Cooks Distance

Feature Selecti...

Learning Curve

Manifold Learni...

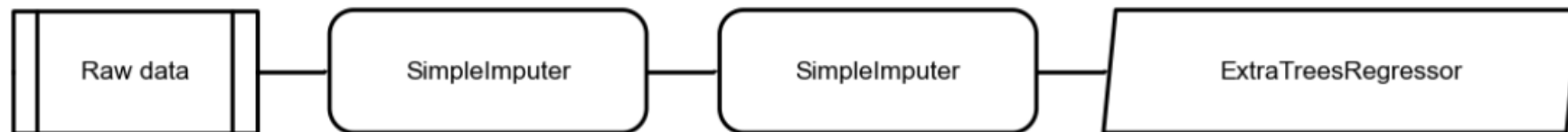
Validation Curve

Feature Importa...

Feature Importa...

Decision Tree

Interactive Res...



예측

In [34]:

1 predict_model(tuned_model)

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
0	Extra Trees Regressor	2.2456	12.1828	3.4904	0.8493	0.1511	0.1185

Out[34]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	prediction_label
410	51.135799	0.0	18.10	0	0.5970	5.757	100.000000	1.4130	24	666	20.200001	2.600000	10.11	15.000000	33.702
85	0.057350	0.0	4.49	0	0.4490	6.630	56.099998	4.4377	3	247	18.500000	392.299988	6.53	26.600000	27.214
280	0.035780	20.0	3.33	0	0.4429	7.820	64.500000	4.6947	5	216	14.900000	387.309998	3.76	45.400002	47.047
422	12.048200	0.0	18.10	0	0.6140	5.648	87.599998	1.9512	24	666	20.200001	291.549988	14.10	20.799999	20.473
199	0.031500	95.0	1.47	0	0.4030	6.975	15.300000	7.6534	3	402	17.000000	396.899994	4.56	34.900002	30.779
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
310	2.635480	0.0	9.90	0	0.5440	4.973	37.799999	2.5194	4	304	18.400000	350.450012	12.64	16.100000	20.511
91	0.039320	0.0	3.41	0	0.4890	6.405	73.900002	3.0921	2	270	17.799999	393.549988	8.20	22.000000	22.671
151	1.496320	0.0	19.58	0	0.8710	5.404	100.000000	1.5916	5	403	14.700000	341.600006	13.28	19.600000	17.677
426	12.247200	0.0	18.10	0	0.5840	5.837	59.700001	1.9976	24	666	20.200001	24.650000	15.69	10.200000	15.066
472	3.568680	0.0	18.10	0	0.5800	6.437	75.000000	2.8965	24	666	20.200001	393.369995	14.36	23.200001	19.422

152 rows × 15 columns

최종 모형 저장

```
In [35]: 1 final_model = finalize_model(tuned_model)
```

```
In [36]: 1 save_model(final_model, 'boston_house_model')
```

Transformation Pipeline and Model Successfully Saved

```
Out[36]: (Pipeline(memory=Memory(location=None),
              steps=[('numerical_imputer',
                    TransformerWrapper(exclude=None,
                                       include=['CRIM', 'ZN', 'INDUS', 'CHAS',
                                              'NOX', 'RM', 'AGE', 'DIS', 'RAD',
                                              'TAX', 'PTRATIO', 'B', 'LSTAT'],
                                       transformer=SimpleImputer(add_indicator=False,
                                                                  copy=True,
                                                                  fill_value=None,
                                                                  missing_values=nan,
                                                                  strategy='mean',
                                                                  verbose='deprecated'))),
                    ('categorical_imputer',
                     Tran...
                    ('actual_estimator',
                     ExtraTreesRegressor(bootstrap=False, ccp_alpha=0.0,
                                          criterion='squared_error', max_depth=None,
                                          max_features=1.0, max_leaf_nodes=None,
                                          max_samples=None,
                                          min_impurity_decrease=0.0,
                                          min_samples_leaf=1, min_samples_split=2,
                                          min_weight_fraction_leaf=0.0,
                                          n_estimators=100, n_jobs=-1,
                                          oob_score=False, random_state=123,
                                          verbose=0, warm_start=False))),
              verbose=False),
          'boston_house_model.pkl')
```

저장된 모형 불러오기

```
In [37]: 1 loaded_model = load_model('boston_house_model')
```

Transformation Pipeline and Model Successfully Loaded

```
In [38]: 1 new_data = df.iloc[20:40,0:-1]
2 predictions = predict_model(loaded_model, data=new_data)
```

```
In [39]: 1 predictions
```

Out[39]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	prediction_label
20	1.25179	0.0	8.14	0	0.538	5.570	98.099998	3.7979	4	307	21.000000	376.570007	21.020000	13.600000
21	0.85204	0.0	8.14	0	0.538	5.965	89.199997	4.0123	4	307	21.000000	392.529999	13.830000	19.600000
22	1.23247	0.0	8.14	0	0.538	6.142	91.699997	3.9769	4	307	21.000000	396.899994	18.719999	15.200000
23	0.98843	0.0	8.14	0	0.538	5.813	100.000000	4.0952	4	307	21.000000	394.540009	19.879999	14.500000
24	0.75026	0.0	8.14	0	0.538	5.924	94.099998	4.3996	4	307	21.000000	394.329987	16.299999	15.600000
25	0.84054	0.0	8.14	0	0.538	5.599	85.699997	4.4546	4	307	21.000000	303.420013	16.510000	13.900000
26	0.67191	0.0	8.14	0	0.538	5.813	90.300003	4.6820	4	307	21.000000	376.880005	14.810000	16.600000
27	0.95577	0.0	8.14	0	0.538	6.047	88.800003	4.4534	4	307	21.000000	306.380005	17.280001	14.800000

감사합니다.

Q & A